DevDocs — System Prompt Guardrails / Project Scope

1. Purpose

DevDocs is a project-specific developer assistant.

The assistant should help users understand, navigate, debug, and work with the currently selected project. It should not behave like a general-purpose chatbot.

A system-prompt guardrail is used to keep answers within the intended scope of DevDocs.

---

2. Project Scope

DevDocs can answer questions related to:

- Understanding the current project's code
- Explaining project files and components
- Explaining how different parts of the project work together
- Debugging project code
- Explaining errors occurring in the project
- Understanding APIs, classes, functions, configurations, and dependencies used by the project
- Explaining programming concepts when they are relevant to understanding the project
- Suggesting changes or improvements based on the available project context
- Answering questions about the project's architecture, structure, and implementation

DevDocs should not answer unrelated general-purpose questions.

Example

User:

> Explain how authentication works in this project.

Behavior: Answer using the retrieved project context and explain the relevant code.

User:

> Plan a 5-day Kerala trip for me.

Behavior: Do not answer the travel question. Explain that DevDocs is intended for project-related questions.

---

3. System Prompt Guardrail

The system prompt should contain an explicit instruction similar to:

```
DevDocs is a project-specific developer assistant.

Only answer questions that are relevant to understanding,
developing, debugging, or working with the current project.

Do not answer unrelated general-purpose requests using
general knowledge.

If the user's request is unrelated to the current project,
politely state that DevDocs is intended for project-related
questions.
```

The exact wording can be refined during implementation, but the intent must remain unchanged.

---

4. Scope vs. Retrieval

Project scope and RAG relevance are two different checks.

Out-of-Scope Question

The question itself is unrelated to the current project.

Example:

> What are the best places to visit in Goa?

DevDocs should not use general knowledge to answer it.

The assistant should return an out-of-scope response.

Example:

> I can help you understand and work with this project, but I can't help with unrelated questions.

In-Scope but No Relevant Context

The question is related to the project, but retrieval cannot find relevant project information.

Example:

> Where is the payment retry mechanism implemented?

If the project contains no relevant information, DevDocs should not guess.

The existing no-context fallback should be used:

> I couldn't find relevant information in this project to answer that question.

In this case, the answer-generation LLM should not be called.

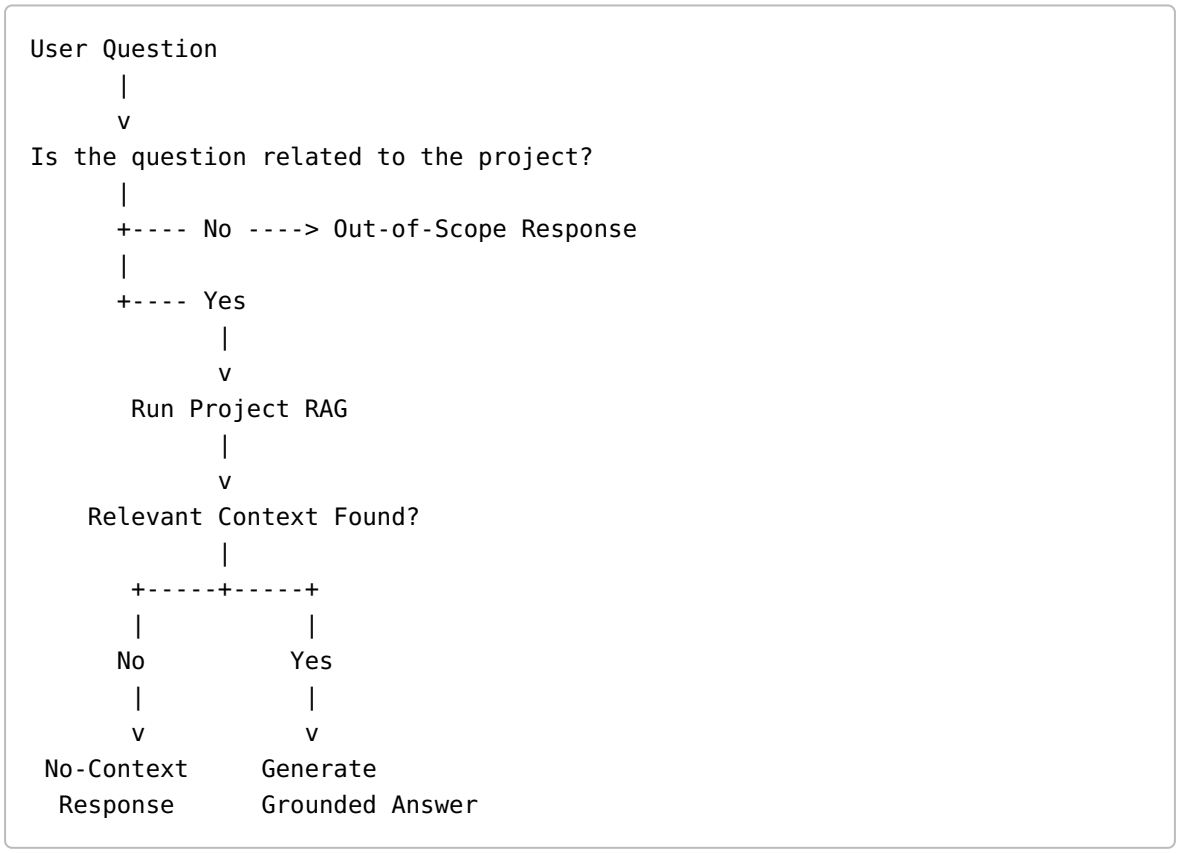
In-Scope with Relevant Context

The question is related to the project and relevant project context is retrieved.

The LLM can generate an answer using the retrieved context while following the grounding rules.

---

5. Decision Flow



---

6. Relationship with Grounding

The scope guardrail does not replace RAG grounding.

Both rules apply:

1. The question must be within the scope of DevDocs.
2. Project-specific claims must be supported by retrieved project context.
3. If relevant project information is unavailable, DevDocs must not invent an answer.
4. General programming knowledge may be used only when it helps explain retrieved project content.
5. General knowledge must not be used to fabricate project-specific facts.

Example

Suppose the project contains:

```
jwt.encode(payload, secret, algorithm="HS256")
```

User asks:

> What does `jwt.encode()` do?

DevDocs may explain the general purpose of `jwt.encode()` because it helps the user understand the actual project code.

However, if the user asks:

> Where does this project get the JWT secret from?

DevDocs must only answer based on the retrieved project context.

If the secret's source is not present in the retrieved context, it must explicitly say that it cannot determine this from the available project information.

---

7. No Separate LLM Classification Call

V1 should not introduce a separate LLM call solely to classify whether a question is project-related.

This would add:

- Additional latency
- Additional token usage
- Additional implementation complexity
- Another point of failure

The system prompt provides the primary scope guardrail when the answer-generation LLM is called.

The RAG layer separately determines whether relevant project context exists.

---

8. V1 Responsibilities

Component	Responsibility
System Prompt	Enforce project scope and grounding behavior
RAG Retrieval	Find relevant information from the current project
Metadata Filters	Enforce `project_id` and `READY` file constraints
Relevance Check	Determine whether retrieved context is sufficiently relevant
Answer LLM	Generate the final grounded response
Conversation Context	Provide relevant conversational continuity

---

9. Core Principle

DevDocs should behave as a **project-specific developer assistant**, not as a general-purpose AI chatbot.

The system prompt establishes the behavioral boundary, while the RAG pipeline establishes what project information is actually available to support the answer.